

Search & Recommendation

Saturday, 16 May 2026 9:20 AM

• Pipeline & Stages

- Ingestion and Pre-Processing
 - Parsing :- Extract raw text + metadata from source formats. Using tools like pdfminer, Apache Tika
 - Cleaning :- Remove boilerplate (headers/footers, navigation links).
 - Normalization :- Lowercasing, Unicode normalization(unicodedata), lemmatization (spaCy).
- Chunking
 - Token-based :- Split by max tokens (e.g., 512–1000), aligns with embedding model's token limits but may cut mid-sentence.
 - sentence-based :- Natural boundaries, preserves meaning but may overshoot token limits.
 - semantic splitting - Split at topic boundaries , LangChain's RecursiveCharacterSplitter, semantic-text-splitter
 - Recursive - start by splitting paragraphs, then sentences, and finally by punctuation
- Tokenization & Embeddings
 - Dense
 - ◆ OpenAI :- text-embedding-3-large/small
 - ◆ BAAI :- bge-large-en
 - ◆ Sentence-Transformers := all-MiniLM-L6-v2
 - Sparse
 - ◆ BM25
 - ◆ ColBERT
 - Multi Model Embedding
 - ◆ Cohere Embed-4, OpenAI text embedding 3 large, GME 7 BN
- Indexing
 - HNSW (Hierarchical Navigable Small World) :- Builds a **multi-layer small-world graph**. It made up of two data structures i.e. Graph and skipped linked list
 - IVF :- Inverted File Index :- Trains **k-means** centroids At query time, find the **closest centroids** (nprobe)
- Retrieval
 - Query Embedding
 - Sparse Retrieval
 - Dense Retrieval
- Fusion
 - Reciprocal Rank Fusion (RRF)
 - Cross encoder
- Ranking
 - Top K Candidate

• Challenges

- Context window fitting
- OOV handling
- Token limits & Out of memory errors
- Retrieval relevance,
 - Hallucination
 - Lost in the middle
- Low Recall
- hybrid search latency
- Freshness
- prompt token limits

• Optimizations

- Simple Optimizations
 - Better Parsers
 - Chunk Sizes
 - Hybrid Search
 - BM-25 & Ensembles
 - MMR
 - Reciprocal Ranking
 - Metadata Filtering
 - Meta Data Filters
- Advanced Optimization
 - Query Expansion (Multi Query Retrieval) : User query → multiple paraphrases/sub-queries.
 - **LangChain MultiQueryRetriever**.
 - **Pyserini RM3 (PRF)**.
 - **ColBERT with PRF**.
 - Reranking
 - BM25/TF-IDF: Keyword-based filtering
 - ◆ **Pyserini + PyTorch** (BM25 + rerank).
 - Cross Encoders .
 - ◆ **HuggingFace Sentence Transformers** (cross-encoders).
 - LLM ReRanker
 - ◆ **LangChain Rerankers** (integrated).
 - Recursive retrieval :- Instead of one-shot retrieval, break queries into **sub-queries** iteratively and then merge the results.
 - **LangChain RecursiveRetriever**.
 - **LlamaIndex AutoMergingRetriever** (progressive retrieval).
 - Embedded tables :- Good for structured sources like tables and csv. Convert **rows/cells into embeddings**
 - **LlamaIndex Structured Retriever**.
 - OpenAI Embeddings + SQL retrievers
 - Embedding fine tuning
 - Multi-Document Agent
 - Quantization
 - Sharding & Routing

• Observability

- Retrial evaluation :- How well system retrieves relevant information
 - Precision :- Of what you **retrieved**, how much was actually relevant?. $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
 - Recall :- Of what was **relevant in the corpus**, how much did you retrieve?. $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
 - Hit Rate :- least one relevant document is present in the top-k in One query
 - MRR :- Reciprocal of rank of the *first* relevant doc. if relevant doc is ranked 3rd → $1/3 = 0.33$.
 - NDCG(Normalized Discounted cumulative Gain) :- Measure the effectiveness of top ranked document

- NDCG (Normalized Discounted Cumulative Gain) :- Measure the effectiveness of top ranked document.
 - MAP :- Mean average Precision , its average of all Precision of docs user watched
 - Generator evaluation
 - BLUE(Precision)/ROUGE(Recall)
 - Grounded-ness/Faithfulness :- *Did the answer's claims come from the provided context?* (no hallucinations)
 - Context Relevance :- Did retrieval fetch the right evidence for this query??
 - Answer Correctness:- Answer compared with Ground Truth?
 - Techniques
 - LLM as a Judge
 - Topic clustering
 - Tools and Frameworks
 - RAGAS
 - LangSmith
 - Arize Phoenix
 - Drift detection & handling the issues
 - Alerts
 - Actions
-
- Knowledge Graph
 - TigerGraph
 - GraphQL
 - Neo4J
 - Cypher Query Language
-
- FAISS
-
- Interview Questions